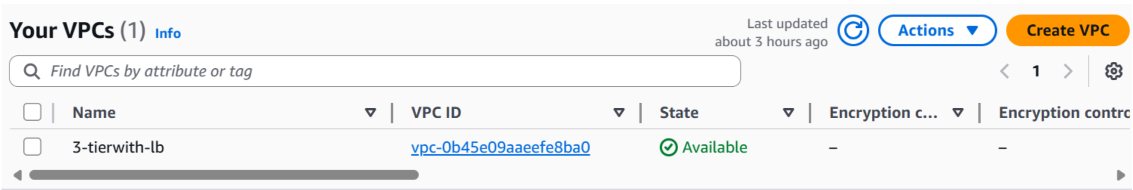
3-Tier Architecture with Application Load Balancer
Project: 3-Tier Architecture with Application Load Balancer (AWS)
Prepared By: Meduri Krishna Teja

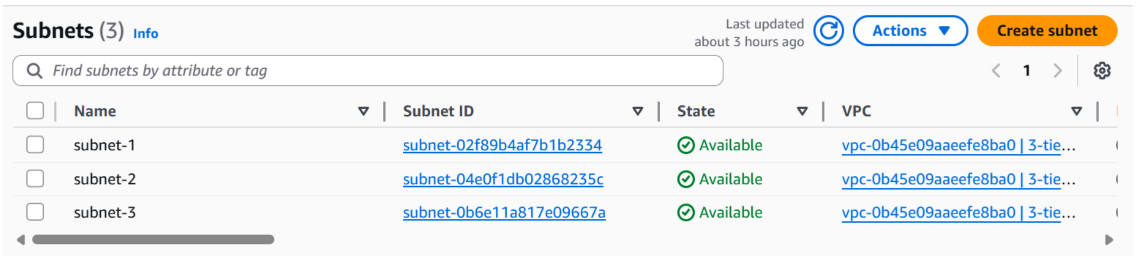
Project Overview:
This project demonstrates the deployment of a **3-Tier Architecture** on AWS using an **Application Load Balancer (ALB)**. The architecture separates the application into three independent layers to improve scalability, security, and maintainability.

Architecture Overview:
- Web Server Layer: Nginx (EC2 instance with Public subnet)
- Application Server Layer: Tomcat (EC2 instance with Public subnet)
- Database Server Layer: MySQL (EC2 instance with Public subnet)
- Application Load Balancer

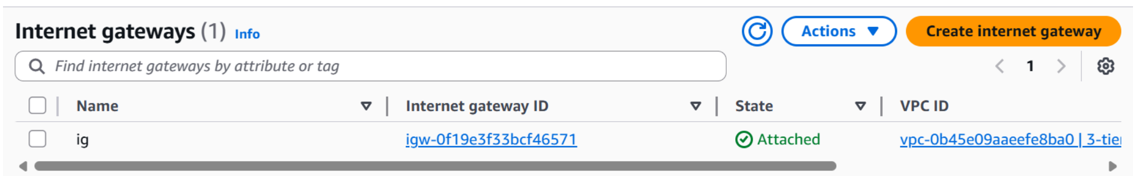
Steps to create a 3-Tier Architecture:
Step1: Create a Virtual Private Cloud (VPC)
A Virtual Private Cloud (VPC) is created with a custom CIDR block. AWS automatically creates the default Route Table, Network ACL, and Security Group for the VPC.



Result:
Custom VPC Created Successfully.
Step2: Create Subnets
Three public subnets are created across different Availability Zones to improve availability and fault tolerance.
- Subnet 1 – Web Server
- Subnet 2 – Application Server
- Subnet 3 – Database Server



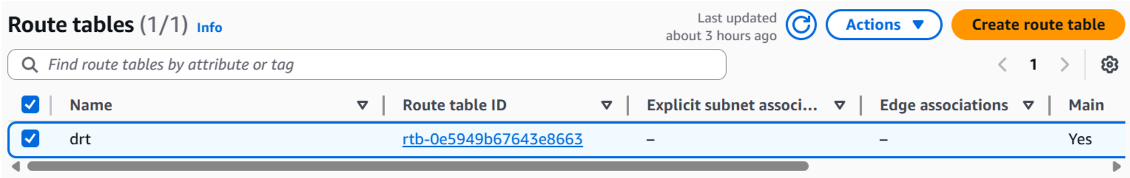
Result:
Three subnets were successfully deployed across multiple Availability Zones.
Step3: Create and Attach an Internet Gateway
An Internet Gateway is created and attached to the VPC, enabling internet access for resources deployed in public subnets.



Result:
Internet Gateway attached successfully.

Step4: Configure the Route Table

The default Route Table is updated with a route (0.0.0.0/0) pointing to the Internet Gateway, allowing outbound internet connectivity.

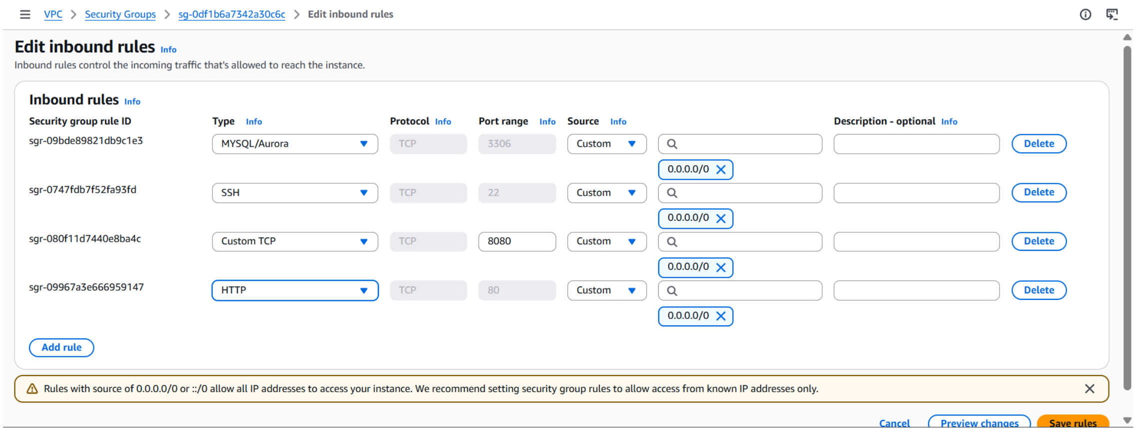


Result:

Public internet access is configured successfully.

Step5: Configure the Security Group

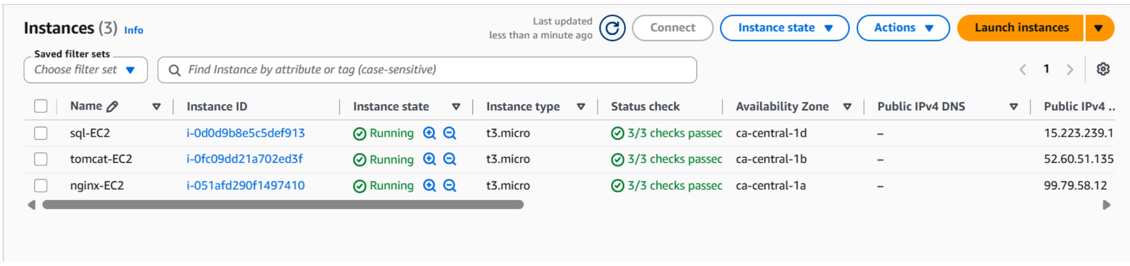
A Security Group is configured with the required inbound rules.



Step6: Launch an EC2 Instance

Three EC2 instances are provisioned using Terraform.

Instances	Purpose
Nginx EC2	Web Server
Tomcat EC2	Application Server
MySQL EC2	Database Server



Result:

All three EC2 instances launched successfully.

Step7: Install and Configure Nginx

Connect to the web server with MobaXterm

root@ip-10-1-1-219:/home/ubuntu#

cmd: apt update

cmd: apt install nginx -y

Now we check if Nginx is installed on our web server.

Nginx is installed on our web server.

Step8: Install Apache Tomcat

Connect to app-ser with MobaXterm

cmd: sudo apt update

cmd: apt install default-jdk -y

cmd: wget https://dlcdn.apache.org/tomcat/tomcat-10/v10.1.49/bin/apache-tomcat-   11.0.14.tar.gz

cmd: ls -lt

cmd: tar -xvzf apache-tomcat-11.0.14.tar.gz

cmd: ls -lt

cmd: mv apache-tomcat-11.0.14 tomcat

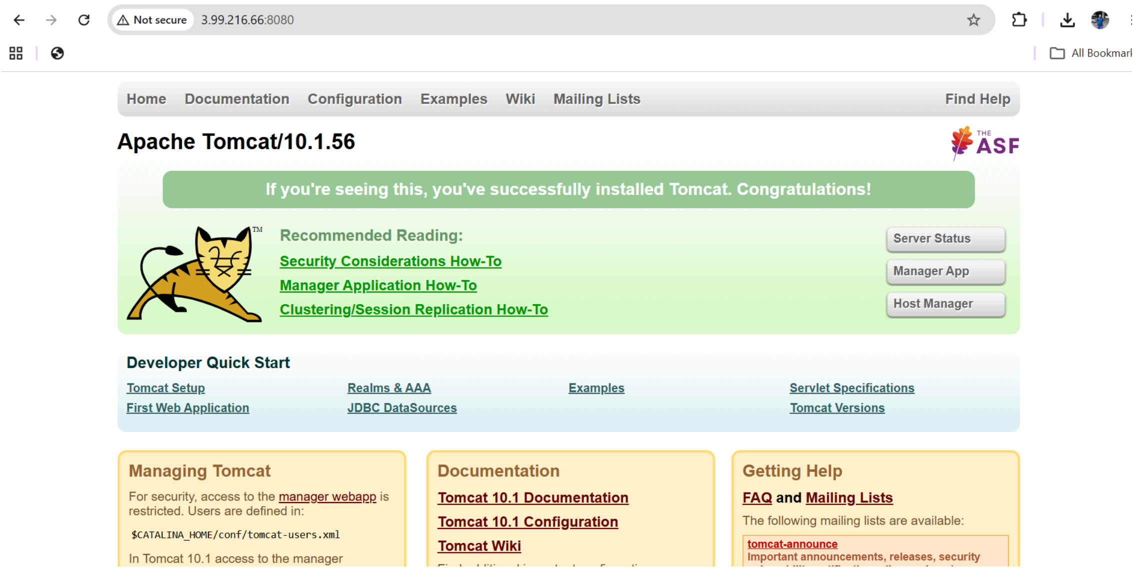
cmd: ls

cmd: cd tomcat/bin/ or cd /home/ubuntu/tomcat/bin

cmd: ls

cmd: ./startup.sh

Now we check if Tomcat is installed or not on our app server.



Tomcat is installed in our app server.

Step9: Install MySQL

Connect to db-ser with MobaXterm

Cmd: apt update

Cmd: apt install mysql-server –y

Cmd: mysql_secure_installation

➢ Edit the MySQL configuration file using a text editor:

vi /etc/mysql/mysql.conf.d/mysqld.cnf

➢ Locate the following line: bind-address = 127.0.0.1

• bind-address = 0.0.0.0

• Save and exit (CTRL + X, then Y, then Enter).

Cmd: systemctl restart mysql

Cmd: systemctl status mysql

```
Escape character is '^]'.
Connection closed by foreign host.
root@ip-10-1-3-39:/home/ubuntu# vi etc/mysql/mysql.conf.d/mysql.cnf
root@ip-10-1-3-39:/home/ubuntu# vi etc/mysql/mysql.conf.d/mysqld.cnf
root@ip-10-1-3-39:/home/ubuntu# vi /etc/mysql/mysql.conf.d/mysqld.cnf
root@ip-10-1-3-39:/home/ubuntu# systemctl restart mysql
root@ip-10-1-3-39:/home/ubuntu# systemctl status mysql
● mysql.service - MySQL Community Server
     Loaded: loaded (/usr/lib/systemd/system/mysql.service; enabled; preset: enabled)
     Active: active (running) since Mon 2026-07-06 06:05:26 UTC; 10s ago
 Invocation: 69cf157935b043c1928faa6f2529b94a
    Process: 15734 ExecStartPre=/usr/share/mysql/mysql-systemd-start pre (code=exited, status=0/
   Main PID: 15751 (mysqld)
     Status: "Server is operational"
      Tasks: 35 (limit: 627)
     Memory: 474.8M (peak: 474.8M)
        CPU: 845ms
     CGroup: /system.slice/mysql.service
             └─15751 /usr/sbin/mysqld

Jul 06 06:05:25 ip-10-1-3-39 systemd[1]: Starting mysql.service - MySQL Community Server...
Jul 06 06:05:26 ip-10-1-3-39 systemd[1]: Started mysql.service - MySQL Community Server.
```

MySQL is installed on our dbserver.

Step10: Verify Server Connectivity

Now connect the web server to the app server.

Open the web server in MobaXterm

Cmd: telnet 3.99.216.66 8080

In this, 3.99.216.66 is the app server's public IP. 8080 is the HTTP port number

```
2. 99.79.58.12 (ubuntu)    4. 15.223.239.124 (ubuntu)    8. 3.99.216.66 (ubuntu)
root@ip-10-1-1-165:/home/ubuntu# telnet 3.99.216.66 8080
Trying 3.99.216.66...
Connected to 3.99.216.66.
Escape character is '^]'.
```

Our web server is successfully connected to the app server.

Step11: Verify Server Connectivity

Now connect the app server to the db server.

Open appserver in MobaXterm

Cmd: telnet 15.223.239.124 3306

In this, 15.223.239.124 is the DB server's public IP. 3306 is the MySQL port number.

```
apache-tomcat-10.1.56/bin/tool-wrapper.sh
apache-tomcat-10.1.56/bin/version.sh
root@ip-10-1-2-147:/home/ubuntu# ls -lt
total 14060
drwxr-xr-x 9 root root     4096 Jul  6 07:51 apache-tomcat-10.1.56
-rw-r--r-- 1 root root 14389873 Jun 17 23:40 apache-tomcat-10.1.56.tar.gz
root@ip-10-1-2-147:/home/ubuntu# mv apache-tomcat-10.1.56 tomcat
root@ip-10-1-2-147:/home/ubuntu# ls
apache-tomcat-10.1.56.tar.gz  tomcat
root@ip-10-1-2-147:/home/ubuntu# cd tomcat/bin/ or cd /home/ubuntu/tomcat/bin
bash: cd: too many arguments
root@ip-10-1-2-147:/home/ubuntu# cd tomcat/bin/
root@ip-10-1-2-147:/home/ubuntu/tomcat/bin# ls
bootstrap.jar       ciphers.sh                 daemon.sh     migrate.bat       shutdown.sh           tool-wrapper.bat
catalina-tasks.xml  commons-daemon-native.tar.gz  digest.bat    migrate.sh        startup.bat           tool-wrapper.sh
catalina.bat        commons-daemon.jar         digest.sh     setclasspath.bat  startup.sh            version.bat
catalina.sh         configtest.bat             makebase.bat  setclasspath.sh   tomcat-juli.jar       version.sh
ciphers.bat         configtest.sh              makebase.sh   shutdown.bat      tomcat-native.tar.gz
root@ip-10-1-2-147:/home/ubuntu/tomcat/bin# ./startup.sh
Using CATALINA_BASE:   /home/ubuntu/tomcat
Using CATALINA_HOME:   /home/ubuntu/tomcat
Using CATALINA_TMPDIR: /home/ubuntu/tomcat/temp
Using JRE_HOME:        /usr
Using CLASSPATH:       /home/ubuntu/tomcat/bin/bootstrap.jar:/home/ubuntu/tomcat/bin/tomcat-juli.jar
Using CATALINA_OPTS:
Tomcat started.
root@ip-10-1-2-147:/home/ubuntu/tomcat/bin# telnet 15.223.239.124 3306
Trying 15.223.239.124...
Connected to 15.223.239.124.
Escape character is '^]'.
```

Step12: Verify Server Connectivity

Now check whether the DB server is connected to the app server or not.

Open dbserver in MobaXterm

Cmd: telnet 3.99.216.66 8080

Step13:Create Target Group

Nginx Target Group (Port 80)

Name	ARN	Port	Protocol	Target type	Load balancer	VPC ID
nginx-tg	arn:aws:elasticloadbalancin...	80	HTTP	Instance	alb	vpc-0b45e09aaeefe8

Step14: Create Target Group

Tomcat Target Group (Port 8080)

tomcat-tg	arn:aws:elasticloadbalancin...	80	HTTP	Instance	alb-1	vpc-0b45e09aaeefe8

Step15: Create Application Load Balancer

Application Load Balancers are deployed across all three public subnets. Each ALB is associated with the default Security Group and configured to forward traffic to its corresponding Target Group.

Name	State	Type	Scheme	IP address type	VPC ID	Availability Zones
alb	Active	application	Internet-facing	IPv4	vpc-0b45e09aaeefe8ba0	3 Availability Zones

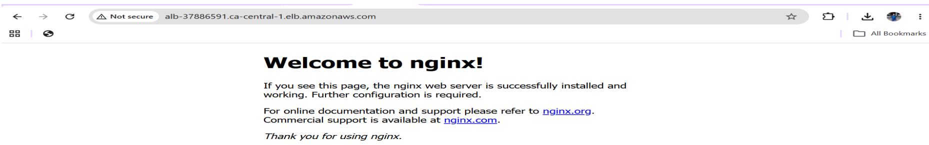
Step16: Create Other Application Load Balancer

Application Load Balancers are deployed across all three public subnets. Each ALB is associated with the default Security Group and configured to forward traffic to its corresponding Target Group.

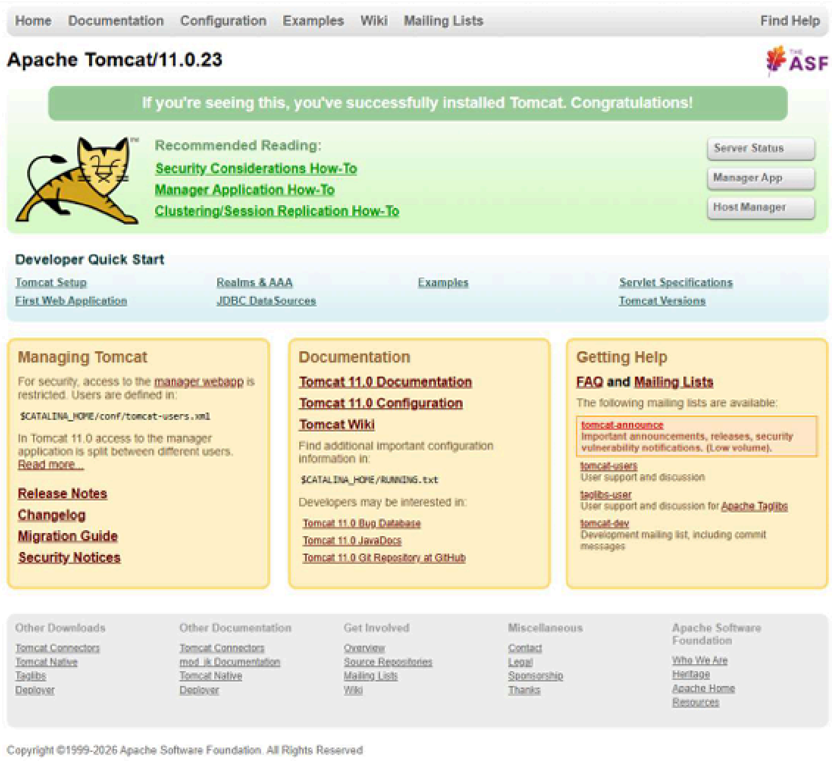
alb-1	Active	application	Internet-facing	IPv4	vpc-0b45e09aaeefe8ba0	3 Availability Zones

The Final result is

Copy the First Application Load Balancer DNS and paste it into Chrome.



Copy the Second Application Load Balancer DNS and paste it into Chrome.



Conclusion:

This project successfully demonstrates the deployment of a scalable 3-tier architecture on AWS using Terraform. The infrastructure includes a custom VPC, public subnets, EC2 instances for the Web, Application, and Database tiers, Application Load Balancers, Target Groups, and Listeners.